

Simple mutex:

```
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox: ~  
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$ gcc -o exe -pthread mutex.c  
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$ cat mutex.c  
#include <pthread.h>  
#include <stdio.h>  
  
#define NUM_THREADS 5  
int shared_data = 0;  
pthread_mutex_t mutex;  
  
void * thread_function(void *arg) {  
    int thread_id = *((int *)arg);  
    pthread_mutex_lock(&mutex);  
    shared_data++;  
    printf("\nthread %d is updating data",thread_id);  
    pthread_mutex_unlock(&mutex);  
    pthread_exit(NULL);  
}  
  
int main() {  
    pthread_t threads[NUM_THREADS];  
    int thread_args[NUM_THREADS];  
    int i;  
  
    for (i = 0; i < NUM_THREADS; i++){  
        thread_args[i] = i;  
        pthread_create(&threads[i], NULL, thread_function, &thread_args[i]);  
    }  
    for (i = 0; i < NUM_THREADS; i++){  
        pthread_join(threads[i], NULL);  
    }  
    printf("\nfinal value of shared data: %d",shared_data);  
    pthread_mutex_destroy(&mutex);  
    return 0;  
}  
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$ ./exe  
  
thread 4 is updating data  
thread 2 is updating data  
thread 0 is updating data  
thread 3 is updating data  
thread 1 is updating data  
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$
```

Semaphores:

```
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox: ~  
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$ gcc -o exe -pthread semaphore.c  
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$ cat semaphore.c  
#include <unistd.h>  
#include <sys/types.h>  
#include <errno.h>  
#include <stdio.h>  
#include <stdlib.h>  
#include <pthread.h>  
#include <string.h>  
#include <semaphore.h>  
  
sem_t mutex;  
int counter;  
void *thread_handler(void *ptr) {  
    int x = *((int*)ptr);  
    printf("sem [INFO] Thread %d Waiting to enter in critical region.\n", x);  
    sem_wait(&mutex);  
  
    printf("sem [INFO] Thread %d Enters in Critical Region.\n", x);  
    printf("sem [INFO] Thread %d Value of Counter is %d.\n", x, counter);  
    printf("sem [INFO] Thread %d Incrementing The Value of counter.\n", x);  
    counter++;  
    printf("sem [INFO] Thread %d New value of counter is: %d.\n", x, counter);  
    printf("sem [INFO] Thread %d Exiting Critical Region.\n", x);  
  
    sem_post(&mutex);  
    pthread_exit(0);  
}  
  
int main() {  
    int i[2] = {0, 1};  
    pthread_t thread_a, thread_b;  
    counter = 0;  
    sem_init(&mutex, 0, 1);  
  
    pthread_create(&thread_a, 0, (void *)thread_handler, (void *)i[0]);  
    pthread_create(&thread_b, 0, (void *)thread_handler, (void *)i[1]);  
  
    pthread_join(thread_a, NULL);  
    pthread_join(thread_b, NULL);  
    sem_destroy(&mutex);  
    return 0;  
}  
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$ ./exe  
sem [INFO] Thread 0 Waiting to enter in critical region.  
sem [INFO] Thread 0 Enters in Critical Region.  
sem [INFO] Thread 0 Value of Counter is 0.  
sem [INFO] Thread 0 Incrementing The Value of counter  
sem [INFO] Thread 0 New value of counter is: 1  
sem [INFO] Thread 0 Exiting Critical Region.  
sem [INFO] Thread 1 Waiting to enter in critical region.  
sem [INFO] Thread 1 Enters in Critical Region.  
sem [INFO] Thread 1 Value of Counter is 1.  
sem [INFO] Thread 1 Incrementing The Value of counter  
sem [INFO] Thread 1 New value of counter is: 2  
sem [INFO] Thread 1 Exiting Critical Region.  
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$
```

Semaphores with signals:

```
May 3 15:18
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox: ~
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$ gcc -o exe -pthread sem_w_signal.c
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$ cat sem_w_signal.c
#include <stdio.h>
#include <pthread.h>
#include <signal.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>

sem_t s;

void handler(int signal_num) {
    sem_post(&s);
    printf("YOK BAAAYE!\n");
    signal(SIGINT, SIG_IGN);
}

void *singsong(void *param) {
    sem_wait(&s);
    printf("I had to wait until your signal released me!\n");
}

int main() {
    int ok = sem_init(&s, 0, 0);
    if (ok == -1) {
        perror("Could not create unnamed semaphore");
        return 1;
    }

    signal(SIGINT, handler);

    pthread_t tid;
    pthread_create(&tid, NULL, singsong, NULL);

    while(1) {
        printf("Thread running\n");
        sleep(1);
    }

    return 0;
}

muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$ ./exe
Thread running
Thread running
^C
OK BAAAYE!
Thread running
I had to wait until your signal released me!
Thread running
^C
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$
```

Sleeping barber:

```
May 3 15:20
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox: ~
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$ gcc -o exe -pthread sleeping_barber.c
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$ cat sleeping_barber.c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define MAX_CUSTOMERS 25
sem_t waitingRoom;
sem_t barberChair;
sem_t barberPillow;
sem_t seatBelt;

int allDone = 0;

void *barber(void *junk) {
    while (!allDone) {
        printf("The barber is sleeping\n");
        sem_wait(&barberPillow);
        if (allDone) {
            printf("The barber is going home for the day.\n");
            break;
        }
        printf("The barber is cutting hair\n");
        sleep(3);
        printf("The barber has finished cutting hair.\n");
        sem_post(&seatBelt);
    }
}

void *customer(void *number) {
    int num = *(int *)number;
    printf("Customer %d leaving for barber shop.\n", num);
    sleep(5);
    printf("Customer %d arrived at barber shop.\n", num);

    sem_wait(&waitingRoom);
    printf("Customer %d entering waiting room.\n", num);

    sem_wait(&barberChair);
    sem_post(&waitingRoom);

    printf("Customer %d waking the barber.\n", num);
    sem_post(&barberPillow);
    sem_wait(&seatBelt);

    sem_post(&barberChair);
    printf("Customer %d leaving barber shop.\n", num);
}
```

```
May 3 15:21
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox: ~
printf("Customer %d waking the barber.\n", num);
sem_post(&barberPillow);
sem_wait(&seatBelt);

sem_post(&barberChair);
printf("Customer %d leaving barber shop.\n", num);
}

int main() {
pthread_t btid, tid[MAX_CUSTOMERS];
int i, numCustomers, numChairs, Number[MAX_CUSTOMERS];

printf("Enter number of customers and chairs (Max 25): \n");
scanf("%d %d", &numCustomers, &numChairs);

sem_init(&waitingRoom, 0, numChairs);
sem_init(&barberChair, 0, 1);
sem_init(&barberPillow, 0, 0);
sem_init(&seatBelt, 0, 0);

pthread_create(&btid, NULL, barber, NULL);
for (i = 0; i < numCustomers; i++) {
    Number[i] = i;
    pthread_create(&tid[i], NULL, customer, (void *)&Number[i]);
}

for (i = 0; i < numCustomers; i++) pthread_join(tid[i], NULL);

allDone = 1;
sem_post(&barberPillow);
pthread_join(btid, NULL);
return 0;
}
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$ ./exe
Enter number of customers and chairs (Max 25):
7
5
The barber is sleeping
Customer 4 leaving for barber shop.
Customer 0 leaving for barber shop.
```

```
May 3 15:22
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox: ~
sem_post(&barberPillow);
pthread_join(btid, NULL);
return 0;
}
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$ ./exe
Enter number of customers and chairs (Max 25):
7
5
The barber is sleeping
Customer 4 leaving for barber shop.
Customer 0 leaving for barber shop.
Customer 6 leaving for barber shop.
Customer 1 leaving for barber shop.
Customer 3 leaving for barber shop.
Customer 2 leaving for barber shop.
Customer 5 leaving for barber shop.
Customer 4 arrived at barber shop.
Customer 0 entering waiting room.
Customer 5 waking the barber.
Customer 2 arrived at barber shop.
Customer 2 entering waiting room.
The barber is cutting hair.
Customer 1 arrived at barber shop.
Customer 1 entering waiting room.
Customer 4 arrived at barber shop.
Customer 3 arrived at barber shop.
Customer 1 entering waiting room.
Customer 4 arrived at barber shop.
Customer 4 entering waiting room.
Customer 4 arrived at barber shop.
The barber has finished cutting hair.
The barber is sleeping.
Customer 5 leaving barber shop.
Customer 2 waiting the barber.
The barber is cutting hair.
Customer 6 entering waiting room.
The barber has finished cutting hair.
Customer 2 leaving barber shop.
The barber is sleeping.
Customer 1 waking the barber.
The barber is cutting hair.
The barber has finished cutting hair.
Customer 1 leaving barber shop.
Customer 3 waking the barber.
The barber is sleeping.
The barber is cutting hair.
The barber has finished cutting hair.
The barber is sleeping.
Customer 6 leaving barber shop.
Customer 4 waiting the barber.
The barber is cutting hair.
The barber has finished cutting hair.
Customer 4 leaving barber shop.
Customer 6 waking the barber.
The barber is cutting hair.
The barber has finished cutting hair.
Customer 6 leaving barber shop.
The barber is going home for the day.
muhammad-affan-rasheed@muhammad-affan-rasheed-VirtualBox:~$
```